Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана (национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ ГОЛОВНОЙ УЧЕБНО-ИССЛЕДОВАТЕЛЬСКИЙ И МЕТОДИЧЕСКИЙ ЦЕНТР ПРОФЕССИОНАЛЬНОЙ РЕАБИЛИТАЦИИ ЛИЦ С ОГРАНИЧЕННЫМИ ВОЗМОЖНОСТЯМИ ЗДОРОВЬЯ

КАФЕДРА СИСТЕМЫ ОБРАБОТКИ ИНФОРМАЦИИ И УПРАВЛЕНИЯ

РАСЧЕТНО-ПОЯСНИТЕЛЬНАЯ ЗАПИСКА

К НАУЧНО-ИССЛЕДОВАТЕЛЬСКОЙ РАБОТЕ

по дисциплине Оперативный анализ данных

по теме «Анализ размера заработной платы для разных специальностей в зависимости от различных факторов на основе датасета Job Dataset»

Студент ИУ5Ц-72Б (Группа) _____ (Подпись, дата) **И.Н. Мефодьев** (И.О. Фамилия)

Руководитель НИР _____ (Подпись, дата) _____ (И.О. Фамилия)

2025 г.

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана (национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

УТВЕРЖДАЮ
Заведующий кафедрой _____
(Индекс)
_____ _____
(И.О.Фамилия)
« _____ » _____ 20 ____ г.

З А Д А Н И Е
на выполнение научно-исследовательской работы

по теме __Анализ размера заработной платы для разных специальностей в зависимости от различных факторов на основе датасета Job Dataset
Студент группы ___ИУ5Ц-72Б Мефодьев Илья Николаевич_____
(Фамилия, имя, отчество)

Направленность НИР (учебная, исследовательская, практическая, производственная, др.)
_____Исследовательская_____
Источник тематики (кафедра, предприятие, НИР) _____НИР_____

График выполнения НИР: 25% к 3 нед., 50% к 9 нед., 75% к 12 нед., 100% к 15 нед.

Техническое задание Спроектировать систему анализа и визуализации по теме «Анализ размера заработной платы для разных специальностей в зависимости от различных факторов провести визуализацию полученных данных посредством языка программирования Python»

Оформление научно-исследовательской работы:
Расчетно-пояснительная записка на 23 листах формата А4.
Перечень графического (иллюстративного) материала (чертежи, плакаты, слайды и т.п.)
_____
_____
_____
_____

Дата выдачи задания «14» сентября 2025 г.

Руководитель НИР _____ _____
(Подпись, дата) (И.О.Фамилия)

Студент _____ **И.Н. Мефодьев**
(Подпись, дата) (И.О.Фамилия)

Примечание: Задание оформляется в двух экземплярах: один выдается студенту, второй хранится на кафедре.

Аннотация

По заданию требуется проанализировать базу данных, очистить её от ненужных данных, также нужно выдвинуть гипотезы, проанализировать их и составить графики по данным гипотезам. Для данного задания была выбрана база данных «Job Dataset». В ходе работы будут использоваться такие библиотеки как: «Pandas» и «Seaborn». Анализ будет проводиться через программу «Visual Studio Code».

Также будет производиться корреляционный анализ, агрегирование данных, оптимизация памяти, удаление дубликатов, очистка данных. Будет осуществлён анализ гипотез по известным данным и построены соответствующие графики.

В работе будут рассмотрены данные о вакансиях из заданного датасета. В конце, будет сделан вывод о проделанной работе с данным датасетом.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ 5

1. Определение данных для анализа 6

2. Формулирование гипотез 6

3. Подготовка данных для работы 7

4. Исследовательский анализ данных 10

Гипотеза 1 – Опыт работы больше влияет на уровень зарплаты, чем уровень образования. 10

Гипотеза 2 – Вакансии, требующие узкоспециализированных технических навыков, предлагают более высокую зарплату, но при добавлении управленческих навыков относительная премия снижается. 12

Гипотеза 3 – В больших компаниях на аналогичных позициях платят в среднем больше, чем в малых. 15

Гипотеза 4 – Зарплатные предложения имеют сезонную зависимость от даты размещения вакансии, при этом пик зарплат приходится на начало года. 17

Корреляционный анализ 20

ЗАКЛЮЧЕНИЕ 22

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ 22

ВВЕДЕНИЕ

Цель работы:

Исследовать базу данных и изучить данные о вакансиях и размере заработной платы, выявить закономерность различными факторами и размером заработной платы. Это позволит соискателям находить вакансии, которые будут наиболее подходить им по условиям труда и размерам оплаты труда.

Задачи:

1. Определение данных.
2. Формулирование гипотез.
3. Загрузка данных в Python.
4. Проверка данных.
5. Очистка данных.
6. Преобразование данных.
7. Выбор данных для анализа.
8. Агрегирование данных.
9. Визуализация данных.
10. Подтверждение или опровержение поставленных гипотез.
11. Корреляционный анализ.
12. Формулирование ограничений и выводов.

1. Определение данных для анализа

Для анализа был выбран DataSet «Job Dataset». Этот набор данных представляет собой полную коллекцию синтетических объявлений о вакансиях для облегчения исследований и анализа в области тенденций рынка труда.

Таблица имеет 1615940 записей. Каждая запись содержит информацию о названии должности, описание вакансии, обязанности, требуемый опыт и квалификацию, местоположение, включая страну, широту и долготу, тип работы, вилку заработной платы, предпочтения к соискателю, обязанности и прочее.

2. Формулирование гипотез

1. Опыт работы больше влияет на уровень зарплаты, чем уровень образования.
2. Вакансии, требующие узкоспециализированных технических навыков, предлагают более высокую зарплату, но при добавлении управленческих навыков относительная премия снижается.
3. В больших компаниях на аналогичных позициях платят в среднем больше, чем в малых.
4. Зарплатные предложения имеют сезонную зависимость от даты размещения вакансии, при этом пик зарплат приходится на начало года.

Обоснование сформулированных гипотез

1. Выдвинуто предположение, что требуемый опыт работы имеет более сильную корреляцию с верхней границей зарплатного диапазона, чем уровень образования.
2. Выдвинуто предположение, что чем уже и сложнее технический навык требуется для работы, тем выше зарплата. Но если к такому узкому навыку добавляют требования к коммуникации или управлению людьми, то зарплата растет уже не так сильно.

3. Считается, что в крупных компаниях уровень зарплаты в среднем выше, чем в малых и средних.
4. Поскольку в новом году начинается и новый период ведения бухгалтерии, то логично предположить, что новые вакансии появляются в начале года чаще, а соответственно и предлагают соискателям больше.

3. Подготовка данных для работы

Для анализа данных было использовано программное обеспечение «Pandas» совместно с «matplotlib».

```
import pandas as pd
import matplotlib as plt
import seaborn as sns
```

Рисунок 1 – Импортирование зависимостей

3.1. Загрузка данных в Python

Для загрузки csv файла с данными в pandas была использована функция read_csv().

```
PATH_TO_CSV = "python\job_descriptions.csv"
data = pd.read_csv(PATH_TO_CSV)
```

Рисунок 2 – Чтение файла

3.2. Проверка данных

Набор данных был проверен на наличие данных по столбцам. Для этого был использован метод info() для объектов dataframe.

Результат представлен на рисунке 3.

```
13  data.info()

PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS

RangeIndex: 1615940 entries, 0 to 1615939
Data columns (total 23 columns):
 #   Column            Non-Null Count    Dtype
---  ------            --------------    -----
 0   Job Id            1615940 non-null  int64
 1   Experience        1615940 non-null  object
 2   Qualifications    1615940 non-null  object
 3   Salary Range      1615940 non-null  object
 4   location          1615940 non-null  object
 5   Country           1615940 non-null  object
 6   latitude          1615940 non-null  float64
 7   longitude         1615940 non-null  float64
 8   Work Type         1615940 non-null  object
 9   Company Size      1615940 non-null  int64
 10  Job Posting Date  1615940 non-null  object
 11  Preference        1615940 non-null  object
 12  Contact Person    1615940 non-null  object
 13  Contact           1615940 non-null  object
 14  Job Title         1615940 non-null  object
 15  Role              1615940 non-null  object
 16  Job Portal        1615940 non-null  object
 17  Job Description   1615940 non-null  object
 18  Benefits          1615940 non-null  object
 19  skills            1615940 non-null  object
 20  Responsibilities  1615940 non-null  object
 21  Company           1615940 non-null  object
 22  Company Profile   1610462 non-null  object
dtypes: float64(2), int64(2), object(19)
memory usage: 283.6+ MB
```

Рисунок 3 – результат data.info()

3.3. Очистка данных

Работа с дубликатами

При помощи метода drop_dublicates() была совершена попытка удалить записи с полностью совпадающим набором по столбцам.

Листинг представлен на рисунке 4.

```
print(f"Всего записей изначально: {data.shape[0]}")

print(f"Очищено дубликатов: {data.shape[0] - len(data.drop_duplicates())}")
```

```
Всего записей изначально: 1615940
Очищено дубликатов: 0
```

Рисунок 4 – Удаление повторяющихся данных

Число очищенных дубликатов говорит о том, что подготовленные данные хорошего качества. Также не требуется как-то очищать данные. А вот

столбец Salary содержит данные вида «$<мин.>K-$<макс.>K». Имеет смысл перевести эти данные в целочисленный формат и разделить на 2 столбца с минимальной и максимальной зарплатой по данной вакансии. Также заметим, что столбец Experience содержит данные вида «<мин.> to <макс.> Years», что тоже стоит привести к 2 разным столбцам.

Листинг деления вилки зарплат на столбцы представлен на рисунке 5.

```
def parse_salary(salary_str):
    """
    Преобразует строку формата "$<мин>K-$<макс>K" в два числа.
    Пример: "$50K-$80K" → (50000, 80000)
    """
    if pd.isna(salary_str):
        return np.nan, np.nan
    try:
        # Убираем пробелы и символы валюты
        salary_str = str(salary_str).strip().replace('$', '').replace(',', '')
        # Разделяем на минимальную и максимальную части
        if '-' in salary_str:
            min_part, max_part = salary_str.split('-')
        else:
            # Если нет диапазона, используем одно значение для обоих
            min_part = max_part = salary_str
        # Обрабатываем минимальную зарплату
        if 'K' in min_part.upper():
            min_salary = int(min_part.upper().replace('K', '')) * 1000
        elif 'M' in min_part.upper():  # На случай, если есть миллионы
            min_salary = int(min_part.upper().replace('M', '')) * 1000000
        else:
            min_salary = int(min_part)
        # Обрабатываем максимальную зарплату
        if 'K' in max_part.upper():
            max_salary = int(max_part.upper().replace('K', '')) * 1000
        elif 'M' in max_part.upper():
            max_salary = int(max_part.upper().replace('M', '')) * 1000000
        else:
            max_salary = int(max_part)
        return int(min_salary), int(max_salary)
    except Exception as e:
        print(f"Ошибка при обработке значения: {salary_str}. Ошибка: {e}")
        return np.nan, np.nan
# Применяем функцию к столбцу Salary
salary_data = data['Salary Range'].apply(parse_salary)
# Создаем новые столбцы
data['Salary_Min'] = salary_data.apply(lambda x: x[0])
data['Salary_Max'] = salary_data.apply(lambda x: x[1])
data['Salary_Avg'] = salary_data.apply(lambda x: (x[0] + x[1]) / 2 if pd.notna(x[0]) and pd.notna(x[1]) else np.nan)
```

Рисунок 5 – Удаление повторяющихся данных

Листинг деления диапазона требуемого опыта на столбцы представлен на рисунке 6

```python
def parse_experience(exp_str):
    """
    Преобразует строку формата "<мин> to <макс> Years" в два числа.
    Пример: "3 to 5 Years" → (3, 5)
    """
    if pd.isna(exp_str):
        return np.nan, np.nan
    try:
        exp_str = str(exp_str).strip().lower()
        exp_str = exp_str.replace('years', '').replace('year', '').strip()# Убираем слово "years" и лишние пробелы
        if 'to' in exp_str:# Обрабатываем разные форматы
            min_part, max_part = exp_str.split('to')
        min_exp = int(min_part.strip())# Преобразуем в числа
        max_exp = int(max_part.strip())
        return min_exp, max_exp
    except Exception as e:
        import re# Пробуем извлечь числа из строки
        numbers = re.findall(r'\d+', exp_str)
        if len(numbers) >= 2:
            return int(numbers[0]), int(numbers[1])
        elif len(numbers) == 1:
            return int(numbers[0]), int(numbers[0])
        else:
            print(f"Не удалось обработать опыт: {exp_str}. Ошибка: {e}")
            return np.nan, np.nan
exp_data = data['Experience'].apply(parse_experience)   # Применяем функцию к столбцу Experience
data['Experience_Min'] = exp_data.apply(lambda x: x[0]) # Создаем новые столбцы
data['Experience_Max'] = exp_data.apply(lambda x: x[1])
print(data.head(5))
```

```
             Job Id     Experience Qualifications Salary Range    location  ... Salary_Min  Salary_Max  Salary_Avg Experience_Min  Experience_Max
0  1089843540111562  5 to 15 Years         M.Tech    $59K-$99K     Douglas  ...      59000       99000     79000.0              5              15
1   398454096642776  2 to 12 Years            BCA   $56K-$116K    Ashgabat  ...      56000      116000     86000.0              2              12
2   481640072963533  0 to 12 Years            PhD   $61K-$104K       Macao  ...      61000      104000     82500.0              0              12
3   688192671473044  4 to 11 Years            PhD    $65K-$91K  Porto-Novo  ...      65000       91000     78000.0              4              11
4   117057806156508  1 to 12 Years            MBA    $64K-$87K    Santiago  ...      64000       87000     75500.0              1              12

[5 rows x 28 columns]
```

Рисунок 6 – создание новых столбцов и таблица результата

4. Исследовательский анализ данных

Гипотеза 1 – Опыт работы больше влияет на уровень зарплаты, чем уровень образования.

Для этого составим гистограммы, которые будут отображать корреляцию между средней зарплатой, и уровнем образования, а также между зарплатой и опытом работы.

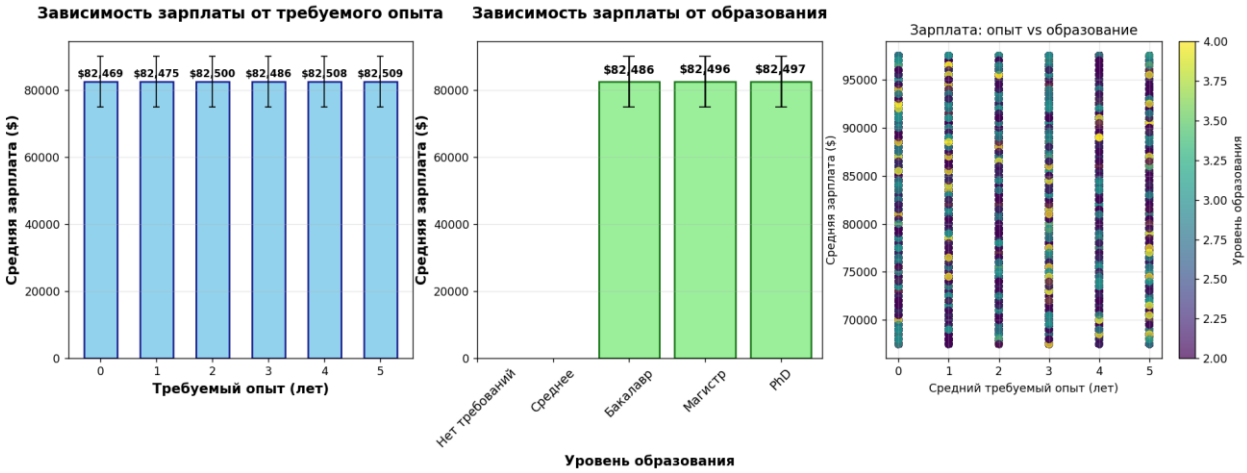

Рисунок 7 – Гистограммы зависимости зарплаты от опыта

Вывод: на основе проведённого анализа гипотеза подтвердилась, имея двухлетний опыт работы, можно претендовать на зарплату большую, чем имея высшее образование.

Так, средняя зарплата для вакансий с требуемым двухлетним опытом работы: **$82,500**

Средняя зарплата для соискателя с образованием PhD: **$82,497**

Код для проверки гипотезы (ввиду большого объёма на рисунке 8 приведена только часть кода для первой диаграммы).

```python
data_an1 = data.copy()
# Создаем числовой показатель для уровня образования
def education_to_numeric(qualification):
    """
    Преобразует квалификацию в числовой показатель.
    Шкала: 0 - без требований, 1 - среднее, 2 - бакалавр, 3 - магистр, 4 - PhD
    """
    if pd.isna(qualification):
        return 0
    qual = str(qualification).lower()
    if any(word in qual for word in ['phd', 'doctorate', 'doctoral']):
        return 4
    elif any(word in qual for word in ['master', 'mba', 'msc', 'mca', 'ma', 'm.tech', 'm.eng', 'm.com']):
        return 3
    elif any(word in qual for word in ['bachelor', 'bca', 'bsc', 'bba', 'ba', 'b.com', 'b.tech', 'b.eng', 'undergraduate']):
        return 2
    elif any(word in qual for word in ['high school', 'secondary', 'diploma', 'associate']):
        return 1
    else:
        return 0  # для других случаев
data_an1.drop(['latitude', 'longitude', 'Company Size'], axis=1) # Применяем функцию
data_an1['Education_Score'] = data_an1['Qualifications'].apply(education_to_numeric)
fig, axes = plt.pyplot.subplots(1, 3, figsize=(18, 6)) # Создаем графики для визуального анализа
# 1. Столбчатая диаграмма зарплаты по опыту
exp_salary_ungrouped = data_an1.groupby('Experience_Min')['Salary_Avg'].agg(['mean', 'std', 'count']).reset_index()
exp_salary_ungrouped = exp_salary_ungrouped.sort_values('Experience_Min')
# Создаем столбчатую диаграмму
bars = axes[0].bar(exp_salary_ungrouped['Experience_Min'],
                   exp_salary_ungrouped['mean'],
                   width=0.6, color='skyblue',
                   edgecolor='navy', linewidth=1.5, alpha=0.9,
                   yerr=exp_salary_ungrouped['std'], capsize=3, error_kw={'elinewidth': 1})
axes[0].set_xlabel('Требуемый опыт (лет)', fontsize=12, fontweight='bold')
axes[0].set_ylabel('Средняя зарплата ($)', fontsize=12, fontweight='bold')
axes[0].set_title('Зависимость зарплаты от требуемого опыта', fontsize=14, fontweight='bold', pad=20)
axes[0].grid(True, alpha=0.3, axis='y')
min_exp = int(exp_salary_ungrouped['Experience_Min'].min())
max_exp = int(exp_salary_ungrouped['Experience_Min'].max())
axes[0].set_xticks(range(min_exp, max_exp + 1))
axes[0].set_xticklabels(range(min_exp, max_exp + 1))
```

Рисунок 8 – Код для вывода диаграммы.

Гипотеза 2 – Вакансии, требующие узкоспециализированных технических навыков, предлагают более высокую зарплату, но при добавлении управленческих навыков относительная премия снижается.

Для этого составим гистограммы, которые будут отображать распределение между средней зарплатой для различных уровней специальности:

1) High_Tech_Only – только технические навыки (>2)

2) High_Tech_with_Mgmt – технические навыки (>2) с примесью менеджмента (>= 1)

3) Mid_Tech_Only – технические навыки (1-2)

4) Mid_Tech_with_Mgmt – технические навыки (1-2) с примесью менеджмента (>= 1)

5) Mgmt_Only – только навыки менеджмента (больше 1)

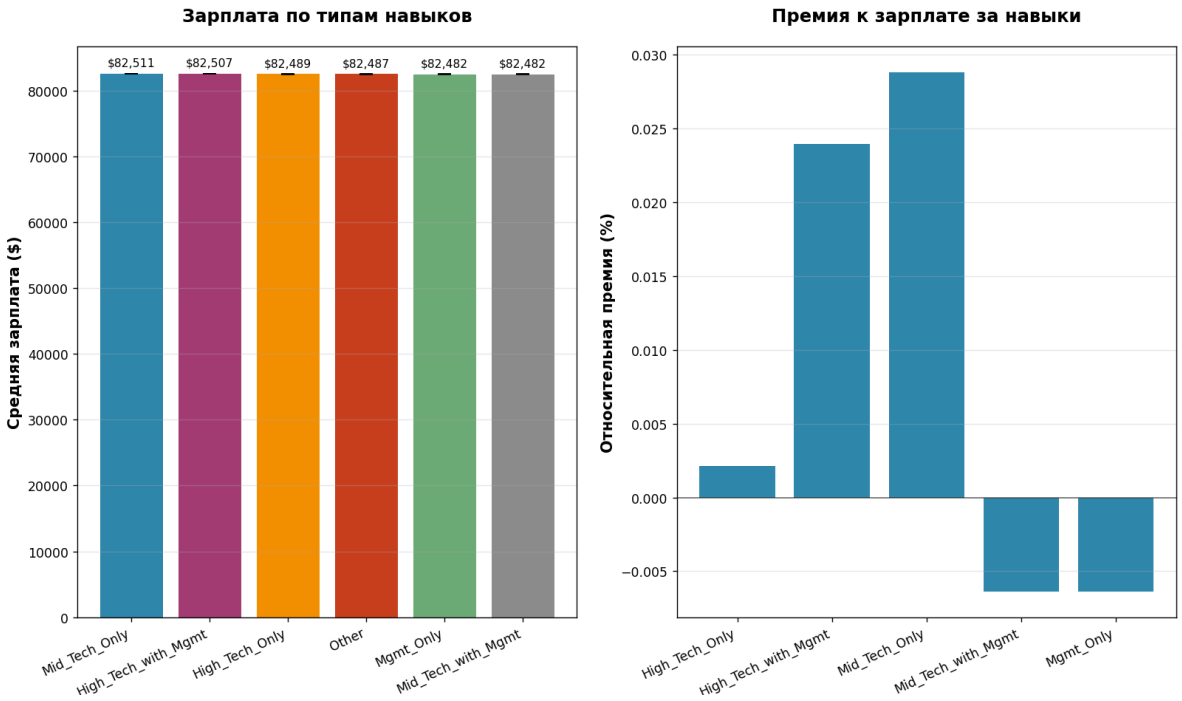

Рисунок 9 – гистограммы зарплат по типам навыков и премии к зарплате за навыки (в сравнении со всеми остальными вакансиями)

Вывод: результаты анализа показывают, что зарплаты менеджеров в среднем ниже, чем у технических специалистов, но при этом для высококвалифицированных технических специалистов добавление мененджерских обязанностей ведёт к повышению зарплаты, а вот для специалистов среднего звена менеджерские качества ведут к уменьшению зарплаты. Таким образом наша гипотеза подтвердилась частично, относительная премия снижается для средних специалистов, а вот у профессионалов премия повышается при добавление управленческих навыков.

Листинг представлен на рисунках 10, 11.

```python
data_an2 = data.copy() # Создаем копию данных для анализа

# 1. Определяем списки навыков
tech_skills_keywords = ['python', 'java', 'javascript', 'c++', 'c#', 'ruby', 'php', 'go', 'rust', 'scala', 'kotlin', 'programming', 'react', 'angular', 'vue', 'node.js', 'django', 'flask', 'spring', 'laravel', 'backend', 'web', 'htm
management_skills_keywords = ['management', 'leadership', 'team lead', 'project management', 'agile', 'scrum', 'stakeholder', 'strategic', 'budget', 'planning', 'mentoring', 'coaching', 'people management', 'resource management', 'r
def find_skills(text, skill_list):# 2. Функция для поиска навыков в тексте
    if pd.isna(text):
        return 0
    text = str(text).lower()
    found_skills = []
    for skill in skill_list:
        if skill in text:
            found_skills.append(skill)
    return len(found_skills)
data_an2['Tech_Skills_Count'] = data_an2['skills'].apply(lambda x: find_skills(x, tech_skills_keywords))# 3. Создаем новые колонки
data_an2['Mgmt_Skills_Count'] = data_an2['skills'].apply(lambda x: find_skills(x, management_skills_keywords))
data_an2['Tech_in_Resp'] = data_an2['Responsibilities'].apply(lambda x: find_skills(x, tech_skills_keywords))# Также проверяем Responsibilities
data_an2['Mgmt_in_Resp'] = data_an2['Responsibilities'].apply(lambda x: find_skills(x, management_skills_keywords))
data_an2['Total_Tech_Skills'] = data_an2['Tech_Skills_Count'] + data_an2['Tech_in_Resp']# Суммируем навыки из обоих источников
data_an2['Total_Mgmt_Skills'] = data_an2['Mgmt_Skills_Count'] + data_an2['Mgmt_in_Resp']
# 4. Классифицируем вакансии по типам навыков
def classify_skills(row):
    tech = row['Total_Tech_Skills']
    mgmt = row['Total_Mgmt_Skills']
    if tech > 2 and mgmt == 0:
        return 'High_Tech_Only'
    elif tech > 2 and mgmt >= 1:
        return 'High_Tech_with_Mgmt'
    elif 1 <= tech <= 2 and mgmt == 0:
        return 'Mid_Tech_Only'
    elif 1 <= tech <= 2 and mgmt >= 1:
        return 'Mid_Tech_with_Mgmt'
    elif tech == 0 and mgmt >= 1:
        return 'Mgmt_Only'
    else:
        return 'Other'

data_an2['Skill_Category'] = data_an2.apply(classify_skills, axis=1)

# 5. Анализ зарплат по категориям навыков
print("\nРАСПРЕДЕЛЕНИЕ ВАКАНСИЙ ПО ТИПАМ НАВЫКОВ:")
category_counts = data_an2['Skill_Category'].value_counts()
for category, count in category_counts.items():
    percentage = (count / len(data_an2)) * 100
    print(f"  {category}: {count} вакансий ({percentage:.1f}%)")

print("\nСРЕДНИЕ ЗАРПЛАТЫ ПО КАТЕГОРИЯМ НАВЫКОВ:")
salary_by_category = data_an2.groupby('Skill_Category')['Salary_Avg'].agg([
    'mean', 'median', 'std', 'count'
```

Рисунок 10 – предварительные расчёты

```python
    base_salary = data_an2[data_an2['Skill_Category'] == 'Other']['Salary_Avg'].mean() # Расчет относительной премии
    if not pd.isna(base_salary):
        premium_tech_only = (group_tech_only.mean() - base_salary) / base_salary * 100
        premium_tech_mgmt = (group_tech_mgmt.mean() - base_salary) / base_salary * 100

fig, axes = plt.pyplot.subplots(1, 2, figsize=(14, 8))# 7. Визуализация
categories = salary_by_category.sort_values('mean', ascending=False).index# График 1: Распределение зарплат по категориям навыков
means = salary_by_category.loc[categories, 'mean']
errors = salary_by_category.loc[categories, 'std'] / np.sqrt(salary_by_category.loc[categories, 'count'])
bars = axes[0].bar(range(len(categories)), means, yerr=errors, capsize=5,
                   color=['#2E86AB', '#A23B72', '#F18F01', '#C73E1D', '#6BAA75', '#8B8B8B'])
axes[0].set_xlabel('Категория навыков', fontsize=12, fontweight='bold')
axes[0].set_ylabel('Средняя зарплата ($)', fontsize=12, fontweight='bold')
axes[0].set_title('Зарплата по типам навыков', fontsize=14, fontweight='bold', pad=20)
axes[0].set_xticks(range(len(categories)))
axes[0].set_xticklabels(categories, rotation=25, ha='right')
axes[0].grid(True, alpha=0.3, axis='y')
for i, bar in enumerate(bars):# Добавляем значения
    height = bar.get_height()
    axes[0].text(bar.get_x() + bar.get_width()/2., height + height*0.01,
                 f'${height:,.0f}', ha='center', va='bottom', fontsize=9)
if 'base_salary' in locals() and not pd.isna(base_salary):# График 3: Относительная премия
    categories_for_premium = ['High_Tech_Only', 'High_Tech_with_Mgmt', 'Mid_Tech_Only', 'Mid_Tech_with_Mgmt', 'Mgmt_Only']
    premiums = []
    for cat in categories_for_premium:
        if cat in salary_by_category.index:
            cat_salary = salary_by_category.loc[cat, 'mean']
            premium = (cat_salary - base_salary) / base_salary * 100
            premiums.append(premium)
        else:
            premiums.append(0)
    axes[1].bar(range(len(categories_for_premium)), premiums, color='#2E86AB')
    axes[1].set_xlabel('Категория навыков', fontsize=12, fontweight='bold')
    axes[1].set_ylabel('Относительная премия (%)', fontsize=12, fontweight='bold')
    axes[1].set_title('Премия к зарплате за навыки', fontsize=14, fontweight='bold', pad=20)
    axes[1].set_xticks(range(len(categories_for_premium)))
    axes[1].set_xticklabels(categories_for_premium, rotation=25, ha='right')
    axes[1].grid(True, alpha=0.3, axis='y')
    axes[1].axhline(y=0, color='black', linestyle='-', linewidth=1, alpha=0.5)    # Добавляем линию нуля
    for i, premium in enumerate(premiums): # Добавляем значения
        axes[1].text(i, premium + (1 if premium >= 0 else -1),
                     f'{premium:+.1f}%', ha='center', va='bottom' if premium >= 0 else 'top',
                     fontsize=9, fontweight='bold')
plt.pyplot.tight_layout()
plt.pyplot.show()
```

Рисунок 11 – Вывод диаграмм.

Гипотеза 3 – В больших компаниях на аналогичных позициях платят в среднем больше, чем в малых.

Для этого составим гистограммы, которые будут отображать зарплаты в разных категориях компаний (малые – < 20000 чел, средние – от 20000 до 50000, большие - >50000)

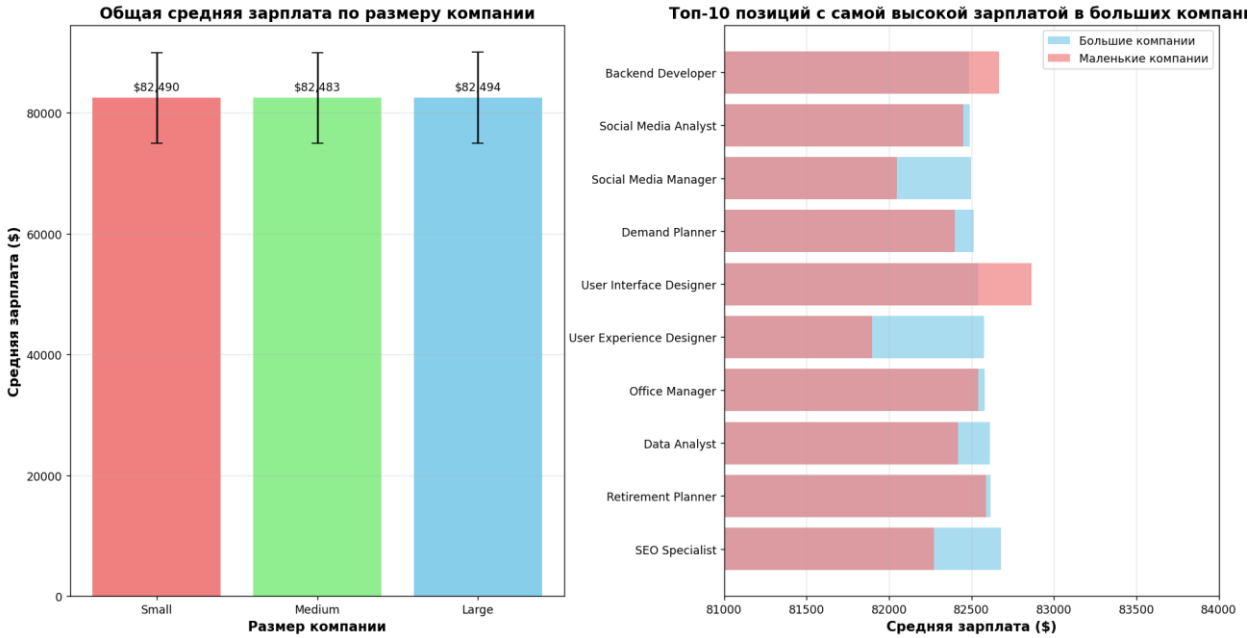

Рисунок 12 – результаты анализа зарплат в зависимости от размера компании

Вывод: Гипотеза в целом подтвердилась, в больших компаниях на 8 из 10 вакансиях платят больше, чем в малых, но, что примечательно, средняя зарплата по всем вакансиям меньше всего в средних компаниях.

Листинг представлен на рисунках 13, 14.

```python
# ГИПОТЕЗА: В больших компаниях на аналогичных позициях платят в среднем больше, чем в маленьких
print("="*80)
print("ГИПОТЕЗА: В больших компаниях на аналогичных позициях (Role) платят")
print("в среднем больше, чем в маленьких компаниях")
print("="*80)

# Создаем копию данных для анализа
data_an3 = data.copy()

# 1. Определяем категории компаний (большие vs маленькие)
print("КЛАССИФИКАЦИЯ КОМПАНИЙ ПО РАЗМЕРУ:")

print(f"Минимальное число сотрудников в компании: {data_an3['Company Size'].min()}")
print(f"Минимальное число сотрудников в компании: {data_an3['Company Size'].max()}")

if 'Company Size' in data_an3.columns:
    def classify_company_size(size):
        if size <= 20000: return "Small"
        if size <= 50000: return "Medium"
        if size > 50000: return "Large"

    data_an3['Company_Category'] = data_an3['Company Size'].apply(classify_company_size)

# Статистика по категориям
category_counts = data_an3['Company_Category'].value_counts()
print("\nРаспределение компаний по категориям:")
for category, count in category_counts.items():
    percentage = (count / len(data_an3)) * 100
    print(f"  {category}: {count} компаний ({percentage:.1f}%)")



# 2. Анализ зарплат по категориям компаний в целом
print("\n📈 ОБЩИЙ АНАЛИЗ ЗАРПЛАТ ПО РАЗМЕРУ КОМПАНИЙ:")

if 'Company_Category' in data_an3.columns:
    # Общая статистика зарплат по категориям компаний
    overall_stats = data_an3.groupby('Company_Category')['Salary_Avg'].agg([
        'count', 'mean', 'median', 'std', 'min', 'max'
    ]).round(0)

    print(overall_stats.sort_values('mean', ascending=False).to_string())
```

Рисунок 13 – Подготовка данных (из-за большого объёма кода приведён лишь фрагмент).

```python
    bars1 = axes[0].bar(range(len(categories)), category_means,
                        yerr=category_stds, capsize=5,
                        color=['lightcoral', 'lightgreen', 'skyblue'])
    axes[0].set_xlabel('Размер компании', fontsize=12, fontweight='bold')
    axes[0].set_ylabel('Средняя зарплата ($)', fontsize=12, fontweight='bold')
    axes[0].set_title('Общая средняя зарплата по размеру компании',
                      fontsize=14, fontweight='bold')
    axes[0].set_xticks(range(len(categories)))
    axes[0].set_xticklabels(categories)
    axes[0].grid(True, alpha=0.3, axis='y')

    # Добавляем значения
    for i, bar in enumerate(bars1):
        height = bar.get_height()
        axes[0].text(bar.get_x() + bar.get_width()/2., height + height*0.01,
                     f'${height:,.0f}', ha='center', va='bottom', fontsize=10)

    # График 2: Топ-10 позиций с наибольшей зарплатой в больших компаниях
    top_10_large = comparison_df.sort_values('Large_Companies', ascending=False).head(10)

    y_pos = range(len(top_10_large))
    axes[1].barh(y_pos, top_10_large['Large_Companies'], color='skyblue', alpha=0.7, label='Большие компании')
    axes[1].barh(y_pos, top_10_large['Small_Companies'], color='lightcoral', alpha=0.7, label='Маленькие компании')

    axes[1].set_yticks(y_pos)
    axes[1].set_yticklabels([role[:25] + '...' if len(role) > 25 else role
                             for role in top_10_large['Role']])
    axes[1].set_xlabel('Средняя зарплата ($)', fontsize=12, fontweight='bold')
    axes[1].set_title('Топ-10 позиций с самой высокой зарплатой в больших компаниях',
                      fontsize=14, fontweight='bold')
    axes[1].legend()
    axes[1].set_xlim(81000,84000)
    axes[1].grid(True, alpha=0.3, axis='x')


    plt.pyplot.tight_layout()
    plt.pyplot.show()
```

Рисунок 14 – Вывод диаграмм.

Гипотеза 4 – Зарплатные предложения имеют сезонную зависимость от даты размещения вакансии, при этом пик зарплат приходится на начало года.

Для этого создадим график, на котором отобразим средние зарплаты по месяцам, выделим начало года, а также экстремумы зарплат:

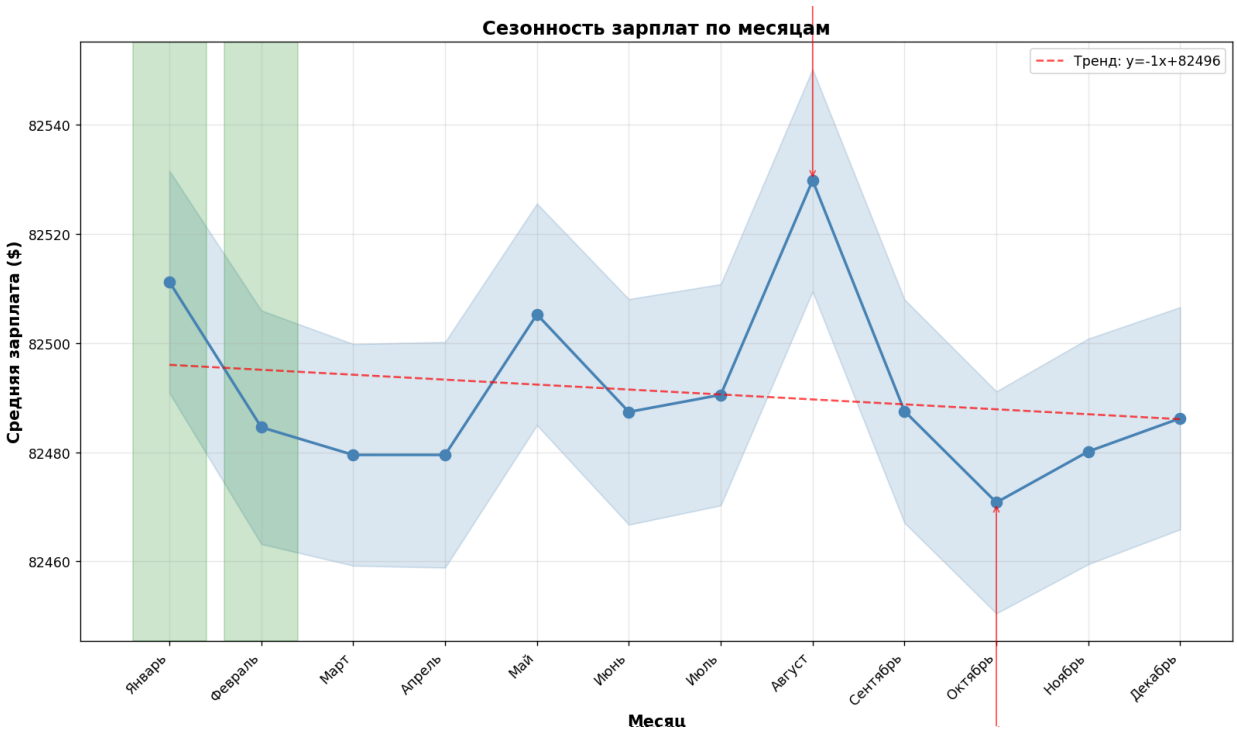

Рисунок 15 – график средних зарплат по месяцам

Вывод: Наша гипотеза не подтвердилась, пик зарплат приходится на август, хотя в январе зарплаты действительно выше среднего.

Листинг представлен на рисунках 16, 17.

```python
data_an4 = data.copy()

# 1. Подготовка данных с датами
print("ПОДГОТОВКА ДАННЫХ С ДАТАМИ:")

# Проверяем наличие колонки с датой
date_column = None
for col in data_an4.columns:
    if 'date' in col.lower() or 'posting' in col.lower():
        date_column = col
        break

if date_column:
    print(f"Найдена колонка с датой: '{date_column}'")

    # Преобразуем в datetime
    try:
        data_an4['Job_Date'] = pd.to_datetime(data_an4[date_column], errors='coerce')

        # Извлекаем компоненты даты
        data_an4['Year'] = data_an4['Job_Date'].dt.year
        data_an4['Month'] = data_an4['Job_Date'].dt.month
        data_an4['Quarter'] = data_an4['Job_Date'].dt.quarter
        data_an4['Week'] = data_an4['Job_Date'].dt.isocalendar().week
        data_an4['Day'] = data_an4['Job_Date'].dt.day
        data_an4['DayOfWeek'] = data_an4['Job_Date'].dt.dayofweek  # 0 = понедельник
        data_an4['Is_StartOfYear'] = data_an4['Month'].isin([1, 2])  # Январь-февраль
        data_an4['Is_EndOfYear'] = data_an4['Month'].isin([11, 12])  # Ноябрь-декабрь

        # Проверяем диапазон дат
        min_date = data_an4['Job_Date'].min()
        max_date = data_an4['Job_Date'].max()
        print(f"Диапазон дат: с {min_date.date()} по {max_date.date()}")
        print(f"Всего лет: {data_an4['Year'].nunique()}")
        print(f"никальные годы: {sorted(data_an4['Year'].dropna().unique())}")

    except Exception as e:
        print(f"Ошибка при обработке даты: {e}")
        date_column = None
else:
    print("Колонка с датой не найдена")
    # Проверим доступные колонки
    print("  Доступные колонки:", [col for col in data_an4.columns if 'date' in col.lower() or 'time' in col.lower()])
```

Рисунок 16 – Подготовка данных.

```python
if 'Month' in data_an4.columns:
    # Группируем по месяцам
    monthly_data = data_an4.groupby('Month')['Salary_Avg'].agg(['mean', 'std', 'count']).sort_index()

    # Создаем данные для графика
    months = monthly_data.index
    month_labels = [month_names[m-1] for m in months]
    mean_salaries = monthly_data['mean']
    std_errors = monthly_data['std'] / np.sqrt(monthly_data['count'])

    axes.plot(range(len(months)), mean_salaries,
              marker='o', linewidth=2, markersize=8, color='steelblue')
    axes.fill_between(range(len(months)),
                      mean_salaries - std_errors,
                      mean_salaries + std_errors,
                      alpha=0.2, color='steelblue')

    # Добавляем линию тренда
    z = np.polyfit(range(len(months)), mean_salaries, 1)
    p = np.poly1d(z)
    axes.plot(range(len(months)), p(range(len(months))),
              "r--", alpha=0.7, label=f'Тренд: y={z[0]:.0f}x+{z[1]:.0f}')

    axes.set_xlabel('Месяц', fontsize=12, fontweight='bold')
    axes.set_ylabel('Средняя зарплата ($)', fontsize=12, fontweight='bold')
    axes.set_title('Сезонность зарплат по месяцам', fontsize=14, fontweight='bold')
    axes.set_xticks(range(len(months)))
    axes.set_xticklabels(month_labels, rotation=45, ha='right')
    axes.grid(True, alpha=0.3)
    axes.legend()

    # Выделяем начало года (январь-февраль)
    for i, month in enumerate(months):
        if month in [1, 2]:
            axes.axvspan(i-0.4, i+0.4, alpha=0.2, color='green', label='Начало года' if i==0 else "")

    # Добавляем аннотации для пиковых месяцев
    for i, (month, salary) in enumerate(zip(months, mean_salaries)):
        if salary == max(mean_salaries) or salary == min(mean_salaries):
            axes.annotate(f'${salary:,.0f}',
                          xy=(i, salary),
                          xytext=(i, salary + (salary*0.05 if salary == max(mean_salaries) else -salary*0.05)),
                          arrowprops=dict(arrowstyle="->", color='red', alpha=0.7),
                          fontsize=10, fontweight='bold', ha='center')

plt.pyplot.tight_layout()
plt.pyplot.show()
```

Рисунок 17 – Вывод диаграммы.

Корреляционный анализ

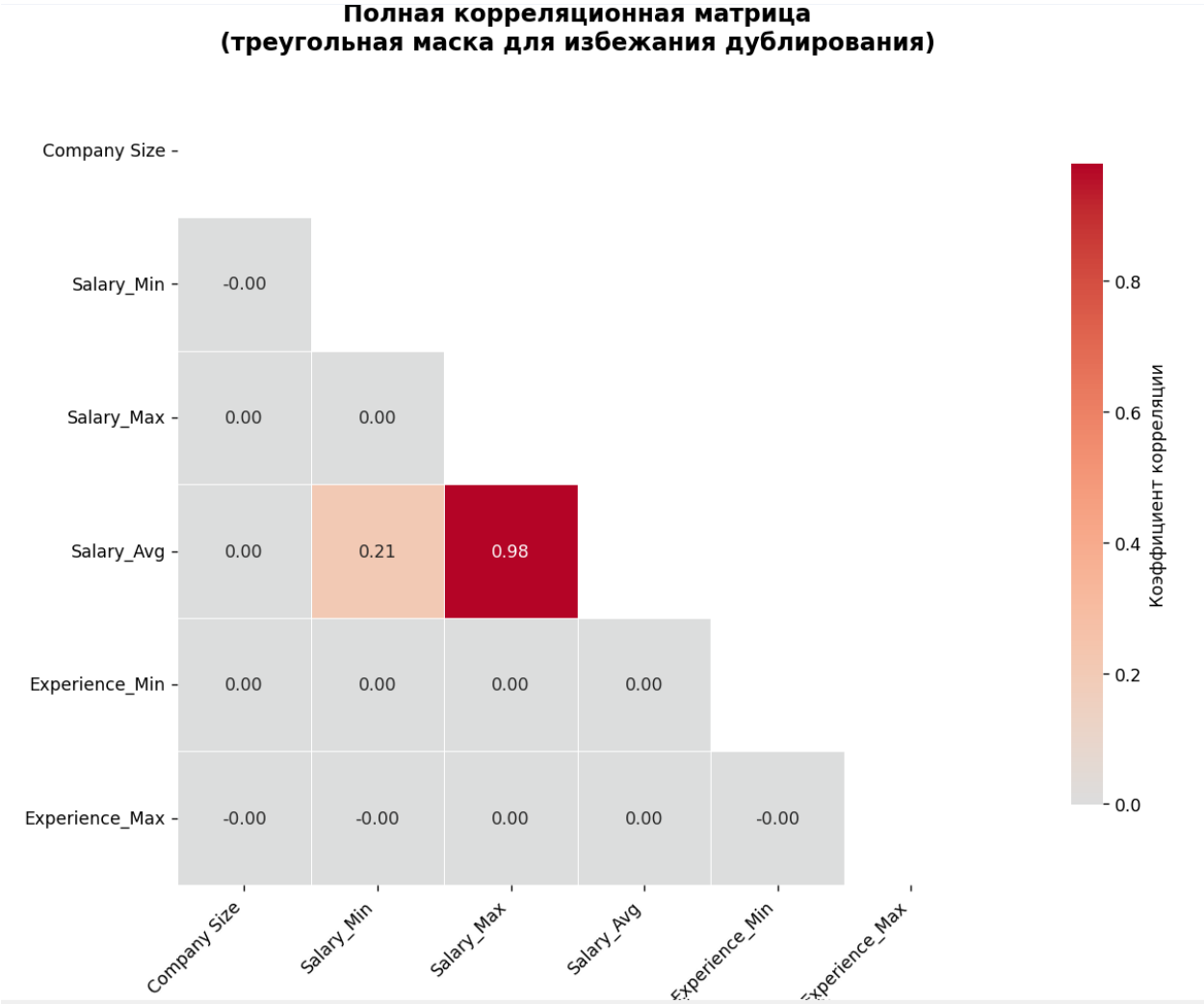

Рисунок 18 - корреляционная матрица

Вывод: корреляций не выявлено, а отмеченные Salary_Avg, Salary_Max и Salary_Min легко объясняются тем, что мы изначально делили эти данные по разным столбцам и высчитывали среднюю зарплату для одной вакансии исходя из вилки зарплат. Практического значения эта корреляция не несёт.

Листинг представлен на рисунке 19:

```python
# Создаем копию данных для анализа
data_corr = data.copy()
numeric_cols = data_corr.select_dtypes(include=[np.number]).columns.tolist()
exclude_keywords = ['job id', 'latitude', 'longitude', 'date']
numeric_cols = [col for col in numeric_cols
                if not any(keyword in col.lower() for keyword in exclude_keywords)]

print(f"  • Найдено {len(numeric_cols)} числовых колонок:")
for i, col in enumerate(numeric_cols[:20], 1):  # Показываем первые 20
    print(f"    {i:2d}. {col}")
if len(numeric_cols) > 20:
    print(f"    ... и еще {len(numeric_cols) - 20} колонок")
corr_data = data_corr[numeric_cols]
correlation_matrix = corr_data.corr()
print(f"  • Размер матрицы: {correlation_matrix.shape[0]}x{correlation_matrix.shape[1]}")
print(f"  • Метод корреляции: Пирсона (линейная корреляция)")
print(f"\n🎨 ВИЗУАЛИЗАЦИЯ КОРРЕЛЯЦИОННОЙ МАТРИЦЫ...")
fig, ax1 = plt.pyplot.subplots(1, 1, figsize=(20, 10))
mask_upper = np.triu(np.ones_like(correlation_matrix, dtype=bool))
sns.heatmap(correlation_matrix,
            mask=mask_upper,
            annot=True,
            fmt='.2f',
            cmap='coolwarm',
            center=0,
            square=True,
            linewidths=0.5,
            cbar_kws={"shrink": 0.8, "label": "Коэффициент корреляции"},
            ax=ax1)
ax1.set_title('Полная корреляционная матрица\n(треугольная маска для избежания дублирования)',
              fontsize=14, fontweight='bold', pad=20)
ax1.set_xticklabels(ax1.get_xticklabels(), rotation=45, ha='right')
ax1.set_yticklabels(ax1.get_yticklabels(), rotation=0)
plt.pyplot.tight_layout()
plt.pyplot.show()
print("Завершение программы...")
```

Рисунок 19 – Вывод корреляционной матрицы

ЗАКЛЮЧЕНИЕ

В ходе научно-исследовательской работы был проведен анализ базы данных «Job Dataset», выявлены важные статистические данные, которые помогли выявить тенденции размещения вакансий и предложения заработных плат на рынке труда.

Для выполнения данной работы было изучено новое программное обеспечение: «Pandas», «Matplotlib» и «Seaborn» на базе языка программирования Python. Также использовался редактор «Visual Studio Code»: в нём происходило написание кода и анализ диаграмм. В ходе работы были получены новые навыки в области анализа данных и произведено знакомство с вышеперечисленными программами.

Была осуществлена оптимизация использования памяти, агрегирование данных, что в дальнейшем позволило провести необходимые в ходе работы исследования и получить ценные знания в предметной области.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1) БУРНАШЕВ Р.А. Методические указания по программному обеспечению «Pandas». - 1 изд. - Казань: КФУ, 2022. - 25 с.

2) Методические указания по программному обеспечению «Seaborn» // [Электронный ресурс]: seaborn.pydata URL: https://seaborn.pydata.org/tutorial.html (дата обращения: 17.10.2025).

3) Методические указания по программному обеспечению «Plotly» // [Электронный ресурс]: habr.com URL: https://habr.com/ru/companies/skillfactory/articles/506974/ (дата обращения: 23.09.2025).

4) «Pandas. Работа с данными» // [Электронный ресурс]: vk.com URL: https://vk.com/wall-159224823_92912 (дата обращения: 15.11.2025).

5) «Python. Визуализация данных: Matplotlib, Seaborn, Mayavi» // [Электронный ресурс]: vk.com URL: https://vk.com/wall-192648009_320 (дата обращения: 23.09.2025).